

北京邮电大学设计实践报告

系统设计名称	课程学习助手 Agent 平台	学 院	网络空间安全学院	指导教师	昌硕
班 级	班内序号	学 号	学生姓名	成绩	
2024211805		2024211667	孙创辉		
2024211805		2024211668	苗新运		
2024211805		2024211669	赵 亮		
系统设计内容	本系统围绕课程学习辅助场景，设计并实现了一个基于 React + FastAPI 的课程学习助手 Agent 平台。系统包括用户登录与个人信息管理、课程与课程资料管理、课程智能问答、知识库管理、学习计划与任务管理、讨论区交流、标签管理、学习数据仪表盘等功能模块；同时设计了前后端分离架构、RESTful API 接口、SQLite/PostgreSQL 数据存储、文件上传与资料检索机制，并结合大模型服务实现面向课程资料的智能问答与学习辅助				
学生设计报告(附页)					
设计成绩评定	指导教师签名： 年 月 日				

课程学习助手 Agent 平台

设计实践报告

摘 要

本项目围绕高校课程学习中资料分散、任务难追踪、提问缺少上下文、笔记结构难维护等问题，设计并实现了课程学习助手 Agent 平台。系统以课程为核心对象，将资料上传与预览、课程对话、学习计划、待办任务、知识库和讨论区组织在统一工作台。前端使用 React 18、TypeScript、Vite、React Router 和 TanStack Query，后端使用 FastAPI、SQLAlchemy、Pydantic 与 JWT，数据层同时兼容 SQLite 本地开发和 PostgreSQL 团队部署。在智能能力方面，系统可读取文本与 PDF 资料，将相关片段和历史消息组合为模型上下文，通过 DeepSeek 的 OpenAI 兼容接口生成带来源提示的回答，并可根据目标、截止日期和每日投入生成阶段任务。

设计过程中重点处理了前后端契约统一、多用户数据隔离、数据库兼容、资料检索与模型降级、路由懒加载、请求缓存、错误边界和多人协作等工程问题。当前版本已通过 TypeScript 生产构建，后端冒烟测试全部通过。实践结果表明，采用模块化前后端架构能够降低功能耦合，基于课程资料的上下文增强能够提高回答的可追溯性，统一的数据缓存与异常处理能够显著改善多页面应用的连续使用体验。

关键词：课程学习；智能助手；FastAPI；React；知识库；检索增强生成；多人协作

项目基本信息

项目项	内容
项目名称	课程学习助手 Agent 平台
系统形态	B/S 架构的前后端分离 Web 应用
主要用户	需要管理课程、资料、任务、笔记和讨论的学习者
前端技术	React 18、TypeScript、Vite、Tailwind CSS、React Router、TanStack Query
后端技术	FastAPI、SQLAlchemy、Pydantic、JWT、pdfplumber、httpx
数据与文件	SQLite / PostgreSQL，server/uploads 文件目录
协作方式	Git + GitHub，多人分支协作，AGENTS.md / CLAUDE.md 统一 Agent 规范

目 录

摘 要.....	2
项目基本信息.....	3
一、课程设计目的.....	7
1.1 项目背景.....	7
1.2 课程设计目标.....	7
1.3 设计范围与边界.....	8
二、需求分析.....	9
2.1 用户与典型场景.....	9
2.2 功能需求.....	9
2.3 非功能需求.....	10
2.4 关键业务流程.....	11
三、详细设计.....	12
3.1 总体架构.....	12
3.2 前端架构与页面组织.....	13
3.3 交互与视觉规范.....	14
3.4 后端模块与 REST 接口.....	15
3.5 认证、权限与文件处理.....	16
3.6 数据库设计.....	16
3.7 SQLite 与 PostgreSQL 兼容.....	18
3.8 课程、资料与智能对话设计.....	18

3.9 学习计划、任务与仪表盘.....	19
3.10 知识库与讨论区.....	20
3.11 部署与运行.....	20
四、难点与亮点.....	22
4.1 多模块数据的一致性.....	22
4.2 资料增强问答的可追溯性.....	22
4.3 数据库双环境与轻量迁移.....	23
4.4 面向真实操作的前端体验.....	23
4.5 多人和多 Agent 协作.....	23
五、设计成果.....	24
5.1 运行界面与功能成果.....	24
5.2 测试环境与方法.....	35
5.3 功能测试用例.....	37
5.4 构建结果分析.....	37
5.5 当前限制与后续计划.....	39
六、设计心得.....	40
6.1 从页面实现到系统设计.....	40
6.2 对 AI 功能的认识.....	40
6.3 对工程协作的认识.....	40
七、团队分工说明.....	42
7.1 协作流程.....	42
7.2 分工评价.....	42
八、AI 伦理声明.....	44

8.1 AI 使用范围.....	44
8.2 数据与隐私原则.....	44
8.3 可靠性与学术诚信.....	44
8.4 公平、透明与可控.....	45
参考资料.....	46
附录 A: 主要目录结构.....	47
附录 B: 提交前检查清单.....	错误!未定义书签。

一、课程设计目的

1.1 项目背景

高校课程学习通常同时涉及课件、讲义、作业、参考文献、课堂问题、复习计划和个人笔记。传统做法往往把这些内容分散在聊天软件、网盘、浏览器收藏夹、待办工具和本地文件夹中，学生需要频繁切换工具，难以形成以课程为中心的连续学习记录。尤其在资料数量增加后，文件可查找但知识不可检索、计划可创建但执行状态不可回顾、提问可得到答案但无法对应课程资料来源等问题逐渐突出。

大语言模型为学习问答和计划拆解提供了新工具，但直接向通用模型提问存在上下文缺失、答案无法追溯和隐私边界不清等风险。因此，本设计不把模型作为孤立聊天窗口，而是将其放入课程、资料、会话和任务的业务链路中，由系统先检索用户拥有的课程资料，再把有限且相关的片段提供给模型。这样既能提升答案针对性，也能保留来源提示和数据权限控制。

1.2 课程设计目标

1. 完成一个可运行的前后端分离系统，形成从登录、课程管理、资料上传到学习问答的完整闭环。
2. 实现学习计划、任务进度、仪表盘和个人中心，使目标、截止时间与每日投入能够被持续追踪。
3. 设计层级知识库、标签、修订、分享与回收站，使零散笔记可以形成可维护的知识结构。
4. 实现帖子、评论、点赞、标签筛选和课程关联，支持学习问题在公共讨论区沉淀。
5. 建立统一认证、资源所有权校验、错误处理、缓存和加载反馈，达到可持续迭代的工程质量。
6. 通过 SQLite 与 PostgreSQL 双环境、环境变量和模块化路由，满足个人本地开发与团队共享部署。
7. 在实践中掌握 React 状态管理、REST API、关系数据库建模、文件处理、LLM 接口和 Git 协作方法。

1.3 设计范围与边界

本次设计聚焦学习者端的核心流程，不实现学校教务系统、教师端成绩管理、实时音视频课堂和支付等外围功能。智能问答采用外部模型接口，不训练自有模型；资料检索采用可解释的全文切片与关键词评分，不宣称达到专用向量数据库的语义检索效果。语雀和 Trilium 目前作为连接入口与可扩展接口，平台主数据仍由本系统维护。

系统的评价重点是功能闭环、架构合理性、接口一致性和使用体验，而不是追求单页视觉效果或复杂动画。前端动效只用于状态过渡、操作反馈和空间关系表达。

二、需求分析

2.1 用户与典型场景

系统当前核心角色为学习者。学习者注册后拥有独立账号空间，可以建立课程、上传资料、发起与课程关联的会话、创建计划和任务、维护知识库以及参与讨论。未认证用户仅可进入登录注册流程；公开分享的知识条目通过独立令牌访问。系统所有私有资源均应以当前用户为边界，避免仅凭前端路由或对象编号访问他人数据。

场景	用户目标	系统响应
开始新学期	建立课程档案	保存课程名称、简介、教师和学期并进入课程详情
整理课件	上传并分类资料	校验课程所有权，保存文件元数据并提供预览或下载
课程提问	得到结合资料的回答	检索课程文件片段，组合历史消息，请求模型并保存回答
制定复习计划	把目标拆成阶段任务	记录截止时间和每日投入，可调用模型生成可执行待办
整理笔记	维护层级知识结构	创建、移动、编辑节点，保存修订，支持标签、分享和回收站
交流问题	发布并跟进讨论	创建帖子、评论、点赞，按标签、课程和关键词检索
复盘进度	快速掌握近期重点	仪表盘聚合任务、课程、资料、计划和最近会话

2.2 功能需求

编号	模块	核心需求	优先级
01	认证与资料	注册、登录、JWT 鉴权、头像上传、昵称修改	高
02	课程	课程增删改查、教师和学期信息、课程详情	高
03	资料	按课程上传、分类、列表、预览、下载和删除	高

编号	模块	核心需求	优先级
04	课程对话	创建会话、关联课程、保存消息、继续最近对话	高
05	智能回答	检索课程资料、生成回答、标注参考来源、错误降级	高
06	学习计划	目标、截止日期、每日投入、计划删除、任务生成	高
07	待办任务	创建、完成切换、截止时间、筛选和删除	高
08	知识库	层级节点、正文、移动、标签、修订、分享、回收站	高
09	讨论区	帖子、评论、标签、点赞、关联课程和相关推荐	中
10	仪表盘	聚合课程、资料、任务、计划、会话和近期截止	中
11	个人中心	个人资料及课程、会话、计划、任务的管理入口	中
12	外部连接	语雀、Trilium 凭据验证和后续扩展入口	低

2.3 非功能需求

类别	需求说明	设计措施
安全性	密码不可明文保存，私有资源必须鉴权	bcrypt 哈希、JWT、后端查询绑定 user_id
可用性	加载、失败、空数据和提交结果应有明确反馈	骨架屏、错误边界、Toast、统一 ApiError
性能	页面切换不应重复加载全部数据	路由懒加载、Query 缓存、hover prefetch、手工分包
可维护性	多人修改时降低耦合与契约漂移	页面、组件、Hook、Router、Schema 分层，协作规范入库
兼容性	本地和共享环境使用不同数据库	SQLAlchemy 抽象，SQLite 特殊参数，PostgreSQL 连接池
可扩展性	模型或知识工具可替换	LLMClient 抽象、环境变量配置、集成路由隔离
可追溯性	智能回答和知识修改应保留上下文	来源提示、消息持久化、知识库修订记录

一致性需求贯穿前后端。前端页面提交数据时必须使用与 Pydantic Schema 一致的字段名和可空规则；后端成功创建资源返回 201，删除成功返回 204，未认证返回 401，无权限或资源不可见返回 403/404。数据变更后由 TanStack Query 使相关缓存失效，而不是强制刷新浏览器。

2.4 关键业务流程

1. 登录流程：客户端提交用户名和密码，后端校验密码哈希，签发带过期时间的 JWT；客户端保存令牌并在后续请求中添加 Authorization 头。
2. 课程资料流程：用户进入自己拥有的课程，选择文件和分类；后端再次校验课程所有权，将文件写入课程目录并记录 Material 元数据。
3. 问答流程：用户在会话中发送问题，后端读取该会话历史与关联课程资料，计算相关片段，调用模型后将用户消息和助手消息同时持久化。
4. 计划流程：用户输入目标、截止日期和每日分钟数，后端保存 Plan；生成任务时要求模型返回严格 JSON，再转为带日期的 Task。
5. 知识库流程：根节点下创建子节点，内容编辑触发保存和修订；删除先进入回收站，恢复时重新加入可见树；分享时生成随机令牌。

三、详细设计

3.1 总体架构

系统采用浏览器/服务器架构。浏览器加载 Vite 构建的 React 单页应用，页面通过统一 API 客户端向 /api 前缀下的 FastAPI 接口发送 JSON 或 multipart/form-data 请求。后端路由负责参数与身份校验，业务逻辑通过 SQLAlchemy 会话访问数据库；资料 and 头像写入 uploads 目录，并通过受控接口或静态挂载访问。智能问答由检索服务和 LLM 客户端协同完成，模型密钥只保存在后端环境变量中。

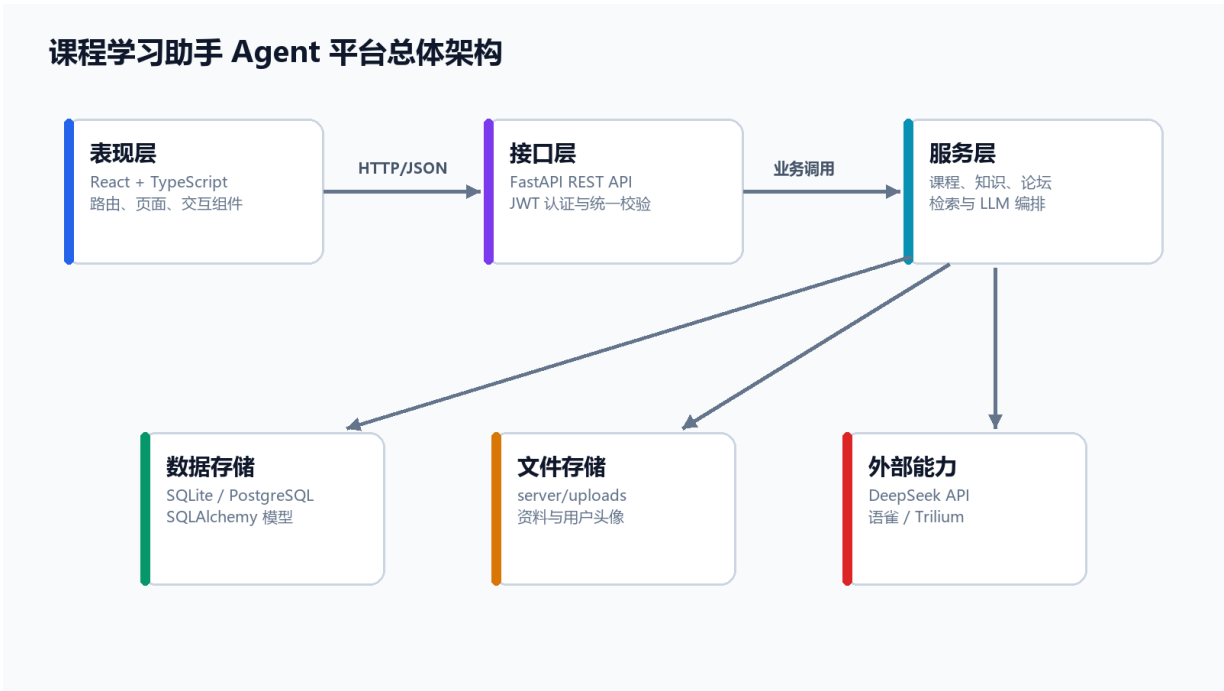


图 3-1 系统总体架构图

层次	主要组件	职责
表现层	页面、Layout、UI 组件、路由	展示信息、收集输入、表达加载与错误状态
前端数据层	api/client.ts、hooks/api.ts	令牌注入、请求封装、缓存、预取和失效

层次	主要组件	职责
接口层	FastAPI Router、Pydantic Schema	REST 路由、参数校验、响应结构和状态码
领域与服务层	LLM、retrieval、业务函数	资料检索、模型编排、计划任务生成
持久化层	SQLAlchemy、SQLite/PostgreSQL、uploads	关系数据、文件内容和事务
外部服务	DeepSeek、语雀、Trilium	模型推理和第三方知识工具连接

3.2 前端架构与页面组织

前端入口使用 `createBrowserRouter` 定义登录页和受保护的主应用。`Layout` 提供侧边导航和账号菜单，`ProtectedRoute` 在页面渲染前验证本地令牌。课程、课程详情、对话、计划、个人中心、仪表盘、知识库和讨论区均采用 `React.lazy` 按路由拆分，`Suspense` 使用统一骨架屏承接加载阶段。路由级错误由 `RouteErrorBoundary` 处理，组件级异常由 `ErrorBoundary` 隔离，避免单个页面错误导致整个应用空白。

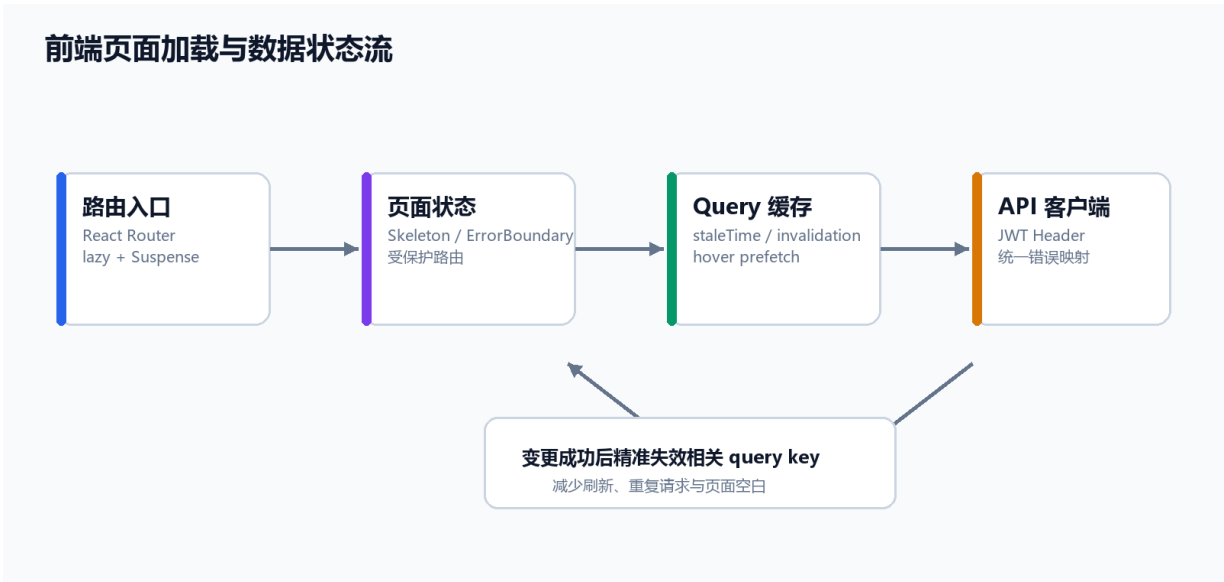


图 3-2 前端页面加载与数据状态流

页面组件不直接散落 fetch 调用。api/client.ts 负责拼接 /api 路径、添加 Bearer Token、处理 204 响应，并把网络错误及 401、403、404、500 等状态映射为统一 ApiError。hooks/api.ts 在此基础上定义 Query 与 Mutation，页面只关心 loading、error、data 和 mutation 状态。创建、更新或删除成功后使精确的 query key 失效，使多个页面能够共享最新数据，同时避免每次切换页面都重新请求。

```
const CourseDetail = lazy(pageLoaders.courseDetail);

function wrap(node: React.ReactNode) {
  return (
    <ErrorBoundary>
      <Suspense fallback={<PageSkeleton lines={5} />}>{node}</Suspense>
    </ErrorBoundary>
  );
}
```

3.3 交互与视觉规范

平台面向高频学习操作，视觉设计采用黑、白、浅灰为主的中性基调，以深色文字、细边框、层级阴影和少量状态色区分信息。按钮按命令强度分为主按钮、次按钮、危险按钮和图标按钮；仅可点击元素使用明显悬停反馈，统计卡片等静态内容不使用位移和强阴影。新增课程、发布帖子、创建计划等操作使用 Dialog 收集信息，避免页面上同时堆叠多个表单。搜索框、筛选器、标签和文件列表保持稳定尺寸，确保动画不会改变布局。

响应式布局以桌面工作台为主要目标，同时通过网格断点调整卡片列数与侧栏状态。图标统一采用 Lucide 或项目已有图标库；不可见的快捷操作必须有 aria-label 或 tooltip。动画主要使用 motion 与 animejs，用于登录卡片入场、侧栏展开、列表增删和按钮状态切换，并尊重 prefers-reduced-motion。

3.4 后端模块与 REST 接口

FastAPI 应用在 `main.py` 中注册各领域 Router。核心接口按 `auth`、`courses`、`materials`、`chat`、`plans`、`tasks`、`dashboard`、`forum` 和 `kb` 分组，统一使用 `/api` 前缀。路由函数通过 `Depends(get_db)` 获取短生命周期数据库会话，通过 `Depends(get_current_user)` 获取当前用户。Pydantic 模型限制请求字段并序列化 ORM 对象，从而使前后端契约可读、可验证，并可在 `/docs` 中自动生成 Swagger 文档。

模块	典型接口	方法	说明
认证	<code>/api/auth/register</code> 、 <code>/login</code> 、 <code>/me</code>	POST/GET/PATCH	注册登录、当前用户与资料维护
课程	<code>/api/courses/</code> 、 <code>/ {course_id}</code>	GET/POST/PUT/DELETE	课程资源 CRUD
资料	<code>/api/materials/course/ {id}</code>	GET/POST	课程资料列表与上传
资料	<code>/api/materials/ {id}/download</code>	GET	鉴权后预览或下载
对话	<code>/api/chat/sessions</code>	GET/POST	会话列表与创建
对话	<code>/sessions/ {id}/messages</code>	GET/POST	读取历史与智能回答
计划	<code>/api/plans/ {id}/generate-tasks</code>	POST	根据目标生成阶段任务
任务	<code>/api/tasks/ {id}/done</code>	PATCH	切换任务完成状态
论坛	<code>/api/posts/</code> 、 <code>/ {id}/vote</code>	GET/POST/PUT/DELETE	帖子、筛选、点赞和相关推荐
知识库	<code>/api/kb/notes</code> 、 <code>/ {id}/content</code>	POST/PUT	创建节点和更新内容
知识库	<code>/api/kb/notes/ {id}/move</code>	PATCH	调整父节点与位置
知识库	<code>/api/kb/trash</code> 、 <code>/ {id}/restore</code>	GET/POST	回收站与恢复
仪表盘	<code>/api/dashboard/</code>	GET	聚合学习状态

接口设计遵循资源语义：列表与详情使用 GET，创建使用 POST，完整修改使用 PUT，局部状态变化使用 PATCH，删除使用 DELETE。资源不存在与不属于当前用户在多数私有场景统一返回 404，以减少对象枚举带来的信息泄露。

3.5 认证、权限与文件处理

密码通过 `passlib` 的 `bcrypt` 上下文生成哈希，数据库只保存 `password_hash`。登录成功后使用 `HS256` 签发包含用户编号和过期时间的 `JWT`。前端把令牌放入 `Authorization: Bearer` 头，后端依赖函数解码后再次查询用户。课程、资料、对话、计划、任务和知识节点的查询都应绑定 `user_id` 或通过所属课程反查用户，不能依赖前端隐藏按钮实现权限。

头像上传限制 `JPEG`、`PNG`、`WebP` 与 `GIF` 类型，并使用随机 `UUID` 作为文件名。课程资料按 `course_{id}` 建立目录，元数据记录原文件名、分类、类型、截止时间和实际路径。下载接口先通过 `Material` 联结 `Course` 校验所有权，再检查磁盘文件是否存在，并使用 `RFC 5987` 编码 `Content-Disposition` 以兼容中文文件名。当前资料上传尚需进一步增加文件大小、危险扩展名和路径规范化限制，这被列入后续安全改进。

3.6 数据库设计

数据层采用 `SQLAlchemy` 声明式模型。`User` 是主要所有权根，`Course`、`Post` 和 `KBNote` 与用户直接关联；`Material` 与 `ChatSession` 归属 `Course`；`ChatMessage` 归属 `ChatSession`；`Plan` 和 `Task` 记录学习目标与执行状态；`Tag` 通过 `PostTag` 与 `KBNoteTag` 被讨论区和知识库复用。知识库额外使用 `Branch` 表描述树关系，`Attribute` 表保存标签式属性，`Revision` 保存历史内容，避免把复杂树结构压缩成单个不可维护字段。

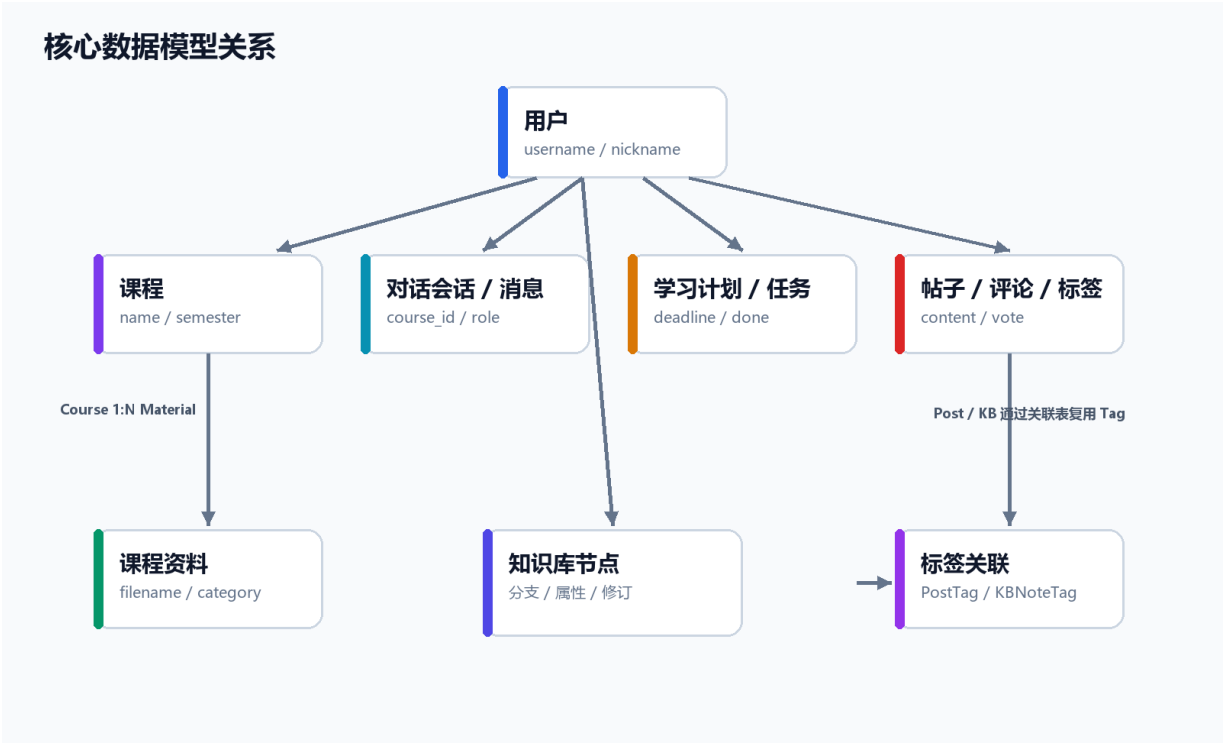


图 3-3 核心数据模型关系图

实体	关键字段	主要关系
User	username、password_hash、nickname、avatar_url	拥有课程、帖子、评论和知识节点
Course	name、intro、teacher、semester、user_id	包含资料和对话会话
Material	course_id、filename、category、content_path	归属单个课程
ChatSession/Message	course_id、session_id、role、content	会话可关联课程并包含多条消息
Plan/Task	goal、deadline、daily_minutes、done	任务可关联计划，也可作为独立待办
Post/Comment	title、content、parent_id、view_count	帖子包含评论、投票和标签
Tag	name、color	通过关联表服务帖子和知识节点
KBNote	note_id、title、content、share_token	拥有分支、属性、修订和标签

3.7 SQLite 与 PostgreSQL 兼容

本地默认 DATABASE_URL=sqlite:///./app.db，创建引擎时关闭多线程限制，便于 FastAPI 请求线程使用；共享部署使用 PostgreSQL 时启用 pool_pre_ping、固定连接池和溢出连接数。SessionLocal 统一提供事务会话，因此 Router 与业务逻辑不依赖具体数据库驱动。配置从 .env 读取，连接串和密钥不进入版本库。

database.py 中的 ensure_schema 会检查已有表和列，对 nickname、avatar_url、知识分享令牌、课程关联、资料分类和截止日期等增量字段进行兼容处理。这种轻量迁移可满足课程实践和小规模迭代，但复杂生产环境仍应引入 Alembic，用可回滚、可审计的版本脚本替代运行时 ALTER TABLE。

3.8 课程、资料与智能对话设计

课程是平台的业务聚合入口。Courses 页面负责筛选和创建课程，CourseDetail 页面汇总授课信息、资料分类、课程讨论与智能入口。上传资料后，前端使课程资料和统计查询失效，列表立即显示新文件。资料预览按类型选择浏览器内联显示或下载，避免为每一种文件格式重复开发预览器。

对话创建时可以关联 course_id。发送消息后，检索服务查询该课程的 Material，文本类文件按 UTF-8 读取，PDF 使用 pdfplumber 提取页面文字；内容按段落切分并过滤过短或符号比例过高的块，过长块截断到固定字符数。查询词在每个块中的词频作为可解释分数，按分数排序后限制每个文件的最大块数和总块数。



图 3-4 课程资料增强问答流程

LLMClient 把系统角色、相关资料和会话历史组合为 OpenAI 兼容 messages，请求 DeepSeek chat/completions。提示词要求回答使用中文，并在引用资料时标注文件名。若关键词未命中，则回退到可读取资料的摘要；不支持解析的格式返回明确说明；HTTP 或网络异常转为可展示的错误文本。此设计强调可用性和可追溯性，但词频检索对同义词不敏感，后续可升级为向量检索与重排序。

3.9 学习计划、任务与仪表盘

Plan 保存目标、截止日期和每日投入分钟数，Task 保存标题、截止时间、完成状态和可选 plan_id。计划生成任务时，后端把目标、截止日期和每日时间发送给模型，并要求返回严格 JSON；解析后按起始日期偏移创建任务。前端把未来七天内截止的计划放入重点队列，避免长期计划过多导致首页信息过载。用户可以手动创建、勾选和删除任务。

Dashboard 接口一次聚合总任务数、完成数、课程数、资料数、计划数、今日任务、近期任务、活跃计划、最近会话和近期资料截止。前端不重复展示多个指向同一页面的按钮，而是把统计卡片与下一步建议组合为 2×2 信息区，确保用户先看到状态，再进入真正需要处理的列表。

3.10 知识库与讨论区

知识库采用类似 Trilium 的节点与分支模型。KBNote 保存节点内容,KBBranch 保存父子关系和排序,因而一个节点的正文与它在树中的位置可以独立维护。创建节点、移动节点、正文保存、标签关联、修订查看、分享、软删除、回收站恢复和全文搜索均有独立 REST 接口。前端对大树使用虚拟列表,编辑正文时保留本地草稿,降低长文编辑因网络异常造成内容丢失的风险。

讨论区支持按最新、热门、精华、课程、标签和关键词筛选。帖子记录浏览数、点赞数和评论数,PostVote 防止用户重复投票,Comment 的 parent_id 支持回复层级。帖子卡片读取作者昵称、用户名和头像,个人资料修改后可由接口返回最新展示信息。课程对话还可以发布为帖子,使个人提问经过整理后进入公共知识沉淀。

3.11 部署与运行

开发环境需要分别启动后端与前端。后端从 server/.env 读取数据库、JWT、模型和上传目录配置,执行 seed 创建演示数据后由 Uvicorn 监听 8000 端口;前端由 Vite 监听 5173 端口并通过代理访问 /api。生产构建使用 tsc -b 先进行类型检查,再由 Vite 输出 dist.vite.config.ts 按 React、路由、Query、动画、Markdown、图标和上传组件拆分 vendorchunk,减小单个包过大警告并利用浏览器长期缓存。

```
# 后端
```

```
cd server
```

```
uvicorn main:app --reload --port 8000
```

```
# 前端
```

```
cd client
```

```
npm run dev
```

```
# 质量检查
```

```
npm run build
```

```
pytest
```

四、难点与亮点

4.1 多模块数据的一致性

平台包含课程、资料、对话、计划、任务、知识库和论坛，多数页面同时引用多个领域的的数据。若每个页面自行维护副本，删除课程后可能出现课程列表已更新但仪表盘仍显示旧数量，修改头像后帖子卡片仍显示旧头像等问题。项目通过统一 Query Key、mutation 后失效和聚合 Dashboard 接口解决跨页面一致性，避免使用 window.location.reload 作为常规刷新手段。

路由懒加载曾带来从侧边栏进入页面空白、刷新后才显示的问题。解决思路是把页面模块加载与数据加载分离：模块由 Suspense 接管，数据由 Query 状态接管，异常由边界组件接管。三种状态各有明确承载组件，页面不再依赖偶然的二次渲染。

4.2 资料增强问答的可追溯性

直接把全部资料发送给模型会超出上下文窗口，也会增加成本和泄露面；只发送问题又会产生通用化回答。项目将课程资料解析为可定位片段后，优先通过 Embedding 模型把用户问题和资料片段映射到同一向量空间，再按余弦相似度召回 Top-K 相关片段，最后把少量上下文交给 LLM 生成回答并标注来源。检索阶段与生成阶段相互解耦，LLM 不直接访问数据库或文件，只基于后端检索到的上下文进行回答。

这一部分的难点在于检索准确性、工程稳定性和演示可用性之间的平衡。系统新增 EmbeddingClient，支持 OpenAI-compatible 的 /embeddings 接口；若未配置外部模型或调用失败，则自动回退到本地轻量哈希向量方案。回退方案会对中文按 2-3 字 ngram 切分、英文按单词切分，再哈希到 4096 维向量并做 L2 归一化，用余弦相似度完成本地检索，保证没有额外 API 时系统也能运行。

相比最初的关键词计数版本，向量检索的亮点是能够把“资料查找”和“答案生成”明确拆开：Embedding 或本地向量化负责把文本转成可比较的向量，相似度排序负责找出处，LLM 负责组织语言。如果后续资料规模扩大，相似度遍历排序还可以替换为 FAISS 或 pgvector，由向量索引负责更高效的 Top-K 检索。

4.3 数据库双环境与轻量迁移

个人开发使用 SQLite 最方便，但团队共享和部署更适合 PostgreSQL。两者在线程参数、连接池、自增主键和 ALTER TABLE 细节上存在差异。项目在创建 engine 时按 URL 分支设置参数，业务层只依赖 SQLAlchemy Session；新增少量字段时先 inspect 再执行兼容 SQL。这样保证本地可快速启动，同时保留远程数据库能力。

轻量迁移的限制也很清晰：复杂约束变更、数据回填和回滚难以可靠表达，PostgreSQL 与 SQLite 的 DDL 差异可能继续扩大。因此报告将 Alembic 迁移、唯一约束和集成测试列为后续工程化重点，而不是把当前兼容逻辑描述成完整方案。

4.4 面向真实操作的前端体验

前端优化不是单纯增加动画。项目逐步把添加类表单收敛为 Dialog，把重复功能从大卡片中移除，把不能点击的统计卡片改为静态，把删除等危险操作使用红色语义；按钮悬停填充只用于尺寸足够的命令按钮，小图标按钮保持稳定可读。课程收藏文件夹、计划卡片堆叠等装饰交互只有在能够进入课程或定位计划时才保留。

性能方面使用路由级代码拆分、manualChunks、TanStack Query 缓存、详情 hover prefetch、知识树虚拟化、编辑草稿自动保存、骨架屏和错误边界。构建产物按 vendor-react、vendor-router、vendor-query、vendor-animation、vendor-markdown 等拆分，当前构建无 Vite 大 chunk 警告。

4.5 多人和多 Agent 协作

多人协作的主要风险不是代码风格差异，而是代理在不了解上下文时覆盖未提交修改、前后端契约只改一侧、提交本地数据库或密钥，以及为了展示效果引入无意义依赖。仓库使用 AGENTS.md 和 CLAUDE.md 固化检查文件、保持范围、同步契约、运行最小测试和 Git 安全规则，使 Codex、Claude Code 与人工开发者在同一质量基线上工作。

五、设计成果

5.1 运行界面与功能成果

系统当前版本已经形成从身份认证、学习总览、课程资料、智能对话到知识沉淀、计划执行和社区讨论的完整运行界面。本节采用 2026 年 7 月 14 日在实际运行环境中重新截取的 14 幅界面，按照用户真实操作路径展示最终设计成果。所有页面共享黑、白、浅灰中性视觉体系，主要操作使用深色强调，状态信息使用少量功能色，既保持学习工具的严肃性，又通过清晰反馈降低操作成本。

5.1.1 身份认证、主导航与外部连接

登录注册页是系统的公开入口。页面左侧使用克制的动态光影和产品主张建立识别度，右侧卡片集中完成登录、注册、头像选择、密码输入与强度提示。表单提交期间会显示加载状态并阻止重复操作；认证成功后保存访问令牌、清除旧用户请求缓存，并进入受保护的業務路由。

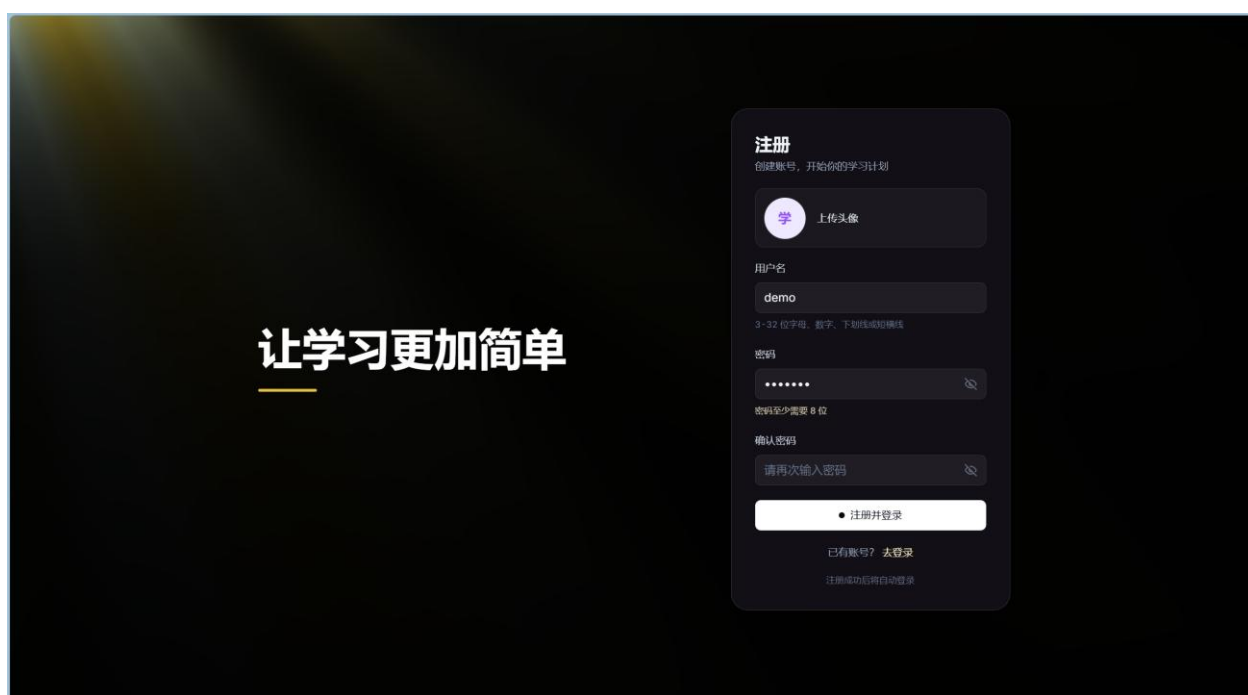


图 5-1 登录注册页（注册模式）

登录后的固定侧边栏提供仪表盘、课程、对话、知识库、学习计划、讨论区、标签管理和个人中心八个入口。底部账号区集中放置当前用户、设置和退出登录，避免将低频账户操作混入业务页面。设置弹窗进一步提供语雀 Token 和 Trilium 服务地址、API Token 的连接验证，为外部知识工具接入保留明确入口。



图 5-2 登录后主导
航与账号菜单



图 5-3 语雀与 Trilium 连接设置

5.1.2 学习总览与课程资料管理

仪表盘承担登录后的学习总览职责。页面把课程数、任务数、完成率和学习计划等统计集中在首屏，并将今日待办、最近课程和学习计划放在同一工作区。用户既可以直接勾选任务，也可以从摘要区域进入对应业务页面，实现从查看状态到继续学习的短路径。

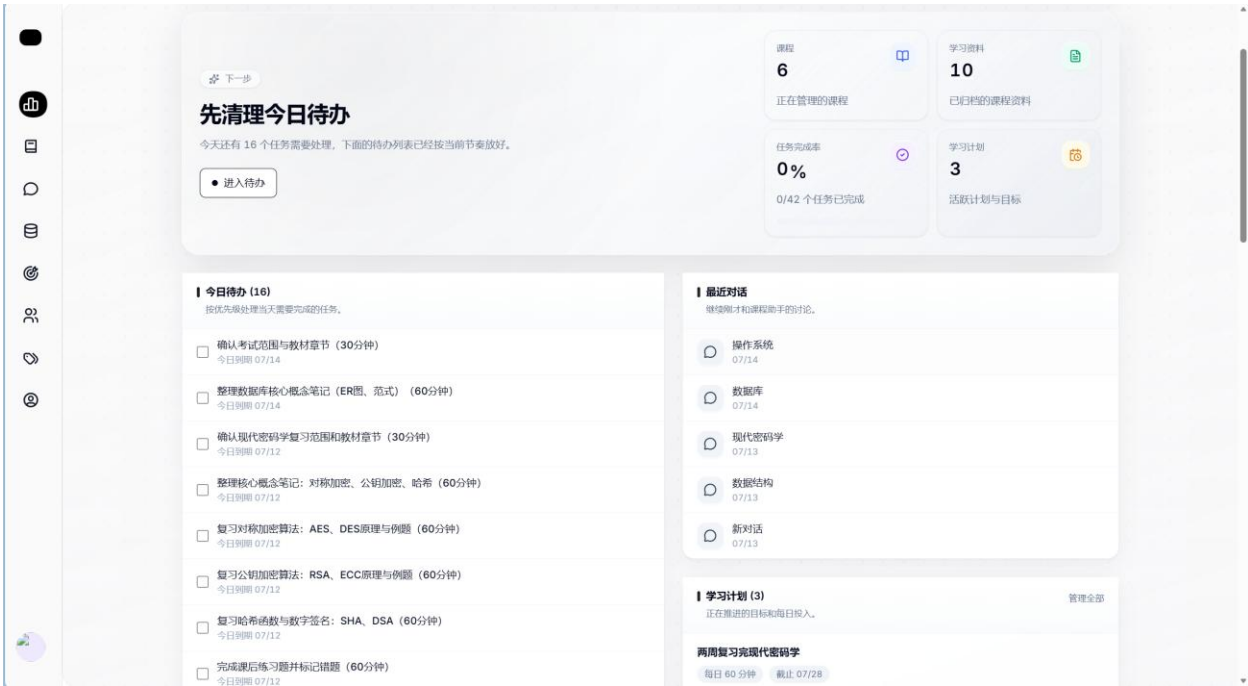


图 5-4 学习仪表盘与今日待办

课程页以课程为学习资源的一级归档单位。顶部概览展示课程数量、资料覆盖和收藏状态，课程库支持学期筛选、关键词搜索、收藏与进入详情。卡片采用稳定网格布局，在宽屏下提高信息密度，在较窄窗口中自动减少列数。

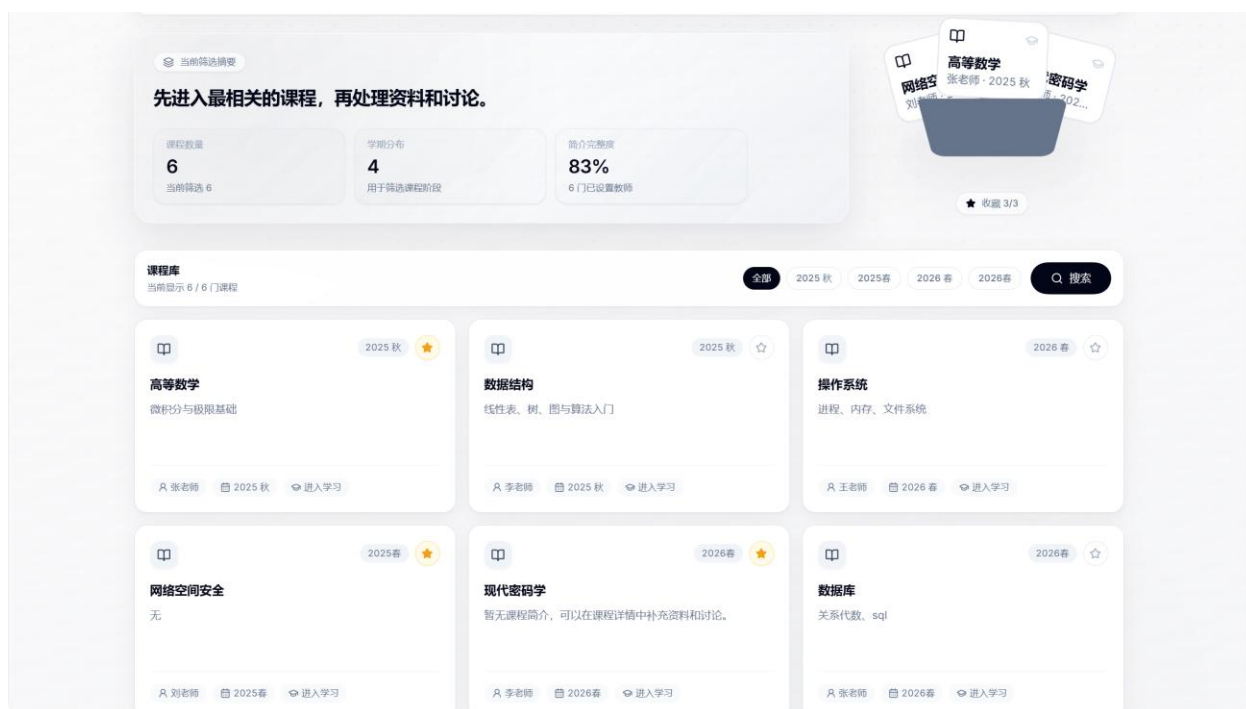


图 5-5 课程总览、收藏与筛选

课程详情页把课程统计、最近资料、资料列表、上传入口、资料分类和相关讨论组织在同一页面。用户可以上传、筛选、预览、下载或删除课程资料，并从课程上下文直接进入智能对话；右侧摘要区域帮助快速判断资料数量、类型和近期学习重点。

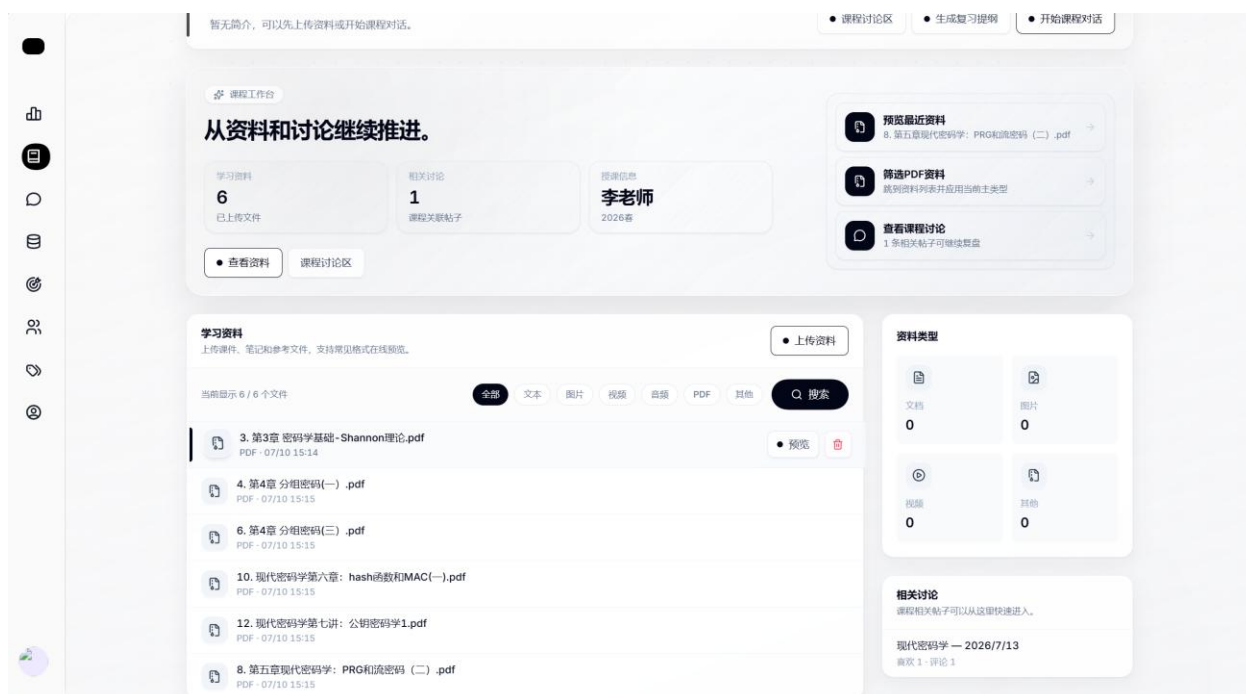


图 5-6 课程详情、资料管理与相关信息

5.1.3 课程对话与知识沉淀

对话页采用三栏工作区：左侧管理历史会话，中间显示提问与 Markdown 回答，右侧呈现当前课程和学习计划上下文。发送问题时，后端根据会话关联课程检索资料片段，再组合历史消息请求模型；回答中的标题、列表、公式和代码由统一 Markdown 渲染器显示。会话可以继续编辑课程关联，也可以整理后发布到知识库或讨论区。



图 5-7 课程上下文增强对话

知识库总览页以知识库根节点为主要对象，展示根节点数量、文档数量、最近整理日期和建议入口。用户可以创建知识库、按条件筛选、搜索已有内容，并从卡片进入对应的层级文档结构。顶部摘要与整理建议用于引导用户优先处理长期未更新或内容较少的知识库。

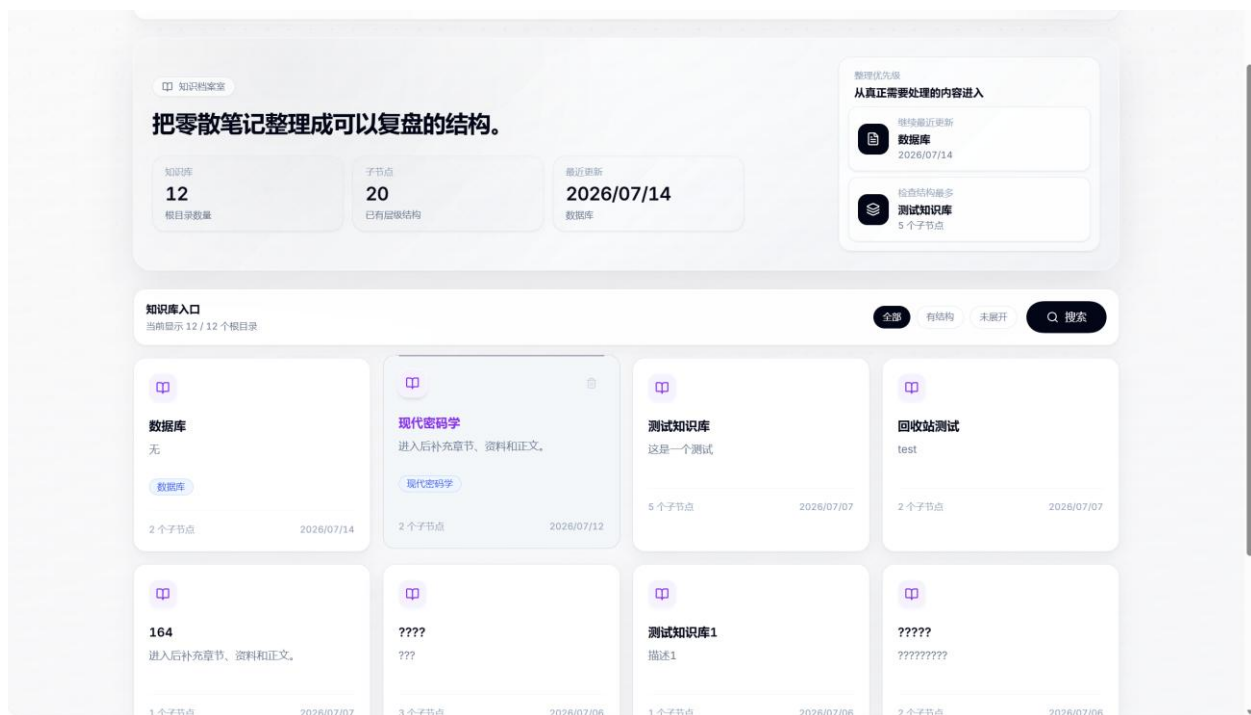


图 5-8 知识库总览与整理入口

知识库详情页由左侧目录树、顶部编辑工具栏和中央文档编辑区构成。目录树支持节点切换和层级组织，编辑区支持富文本与 Markdown 模式、标题格式、链接、图片和表格等操作。保存状态、本地草稿和修订机制共同降低长文编辑中的内容丢失风险，大量节点则通过虚拟化树减少渲染开销。

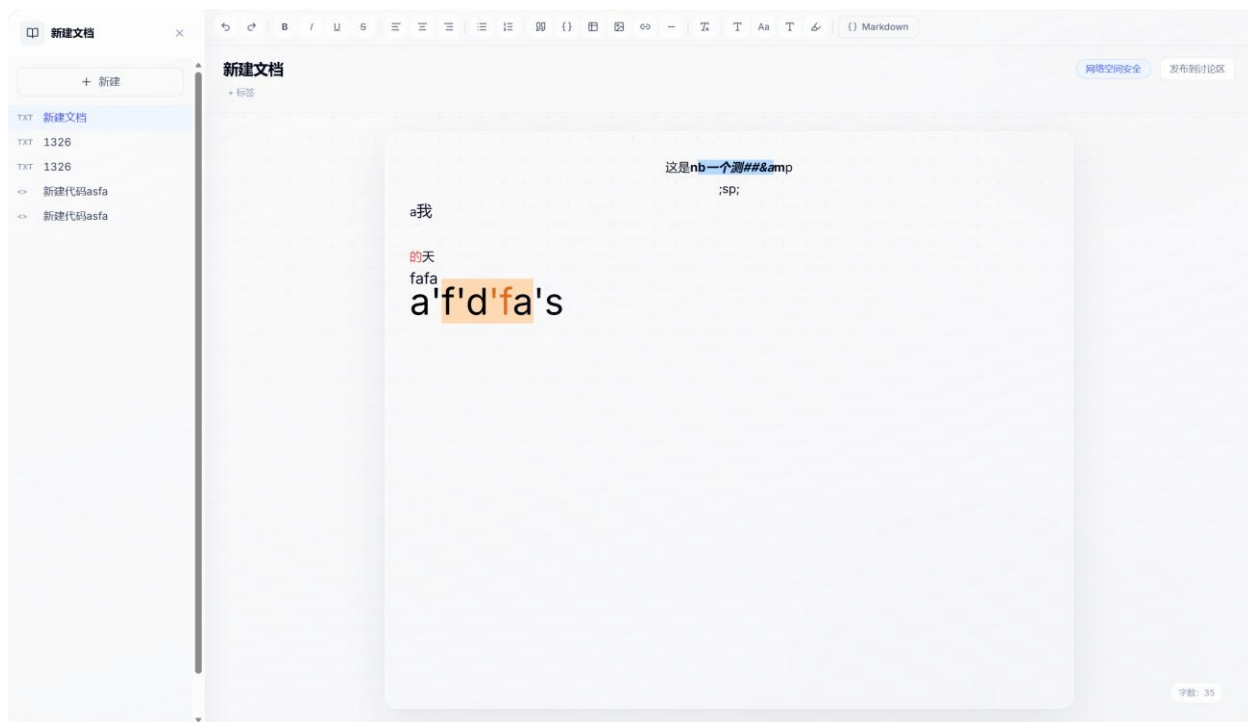


图 5-9 知识库目录树与文档编辑器

5.1.4 学习计划与讨论交流

学习计划页把整体完成率、阶段任务数量、每日投入时间和最近截止时间放在概览区，右侧突出未来七天内需要关注的计划。下方计划卡片展示目标、状态、截止日期和每日分钟数，用户可以创建计划、生成阶段任务、查看进度或删除计划，使长期目标能够被拆解为可执行事项。

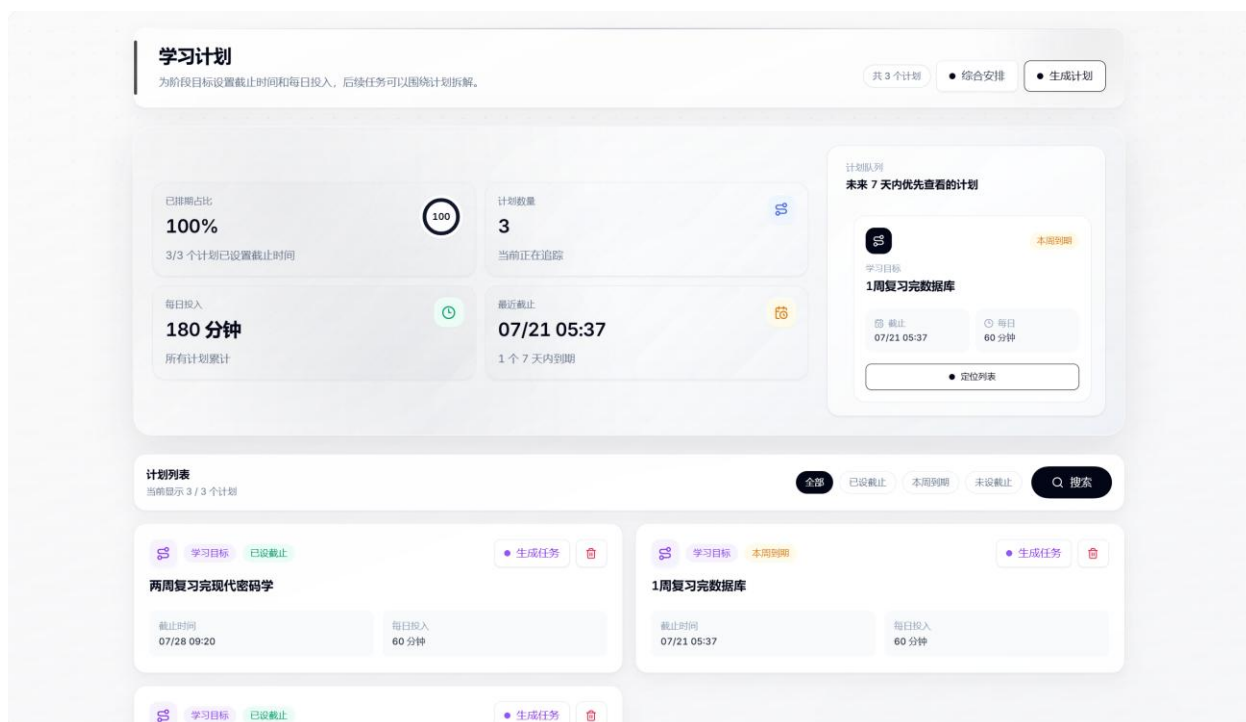


图 5-10 学习计划进度与阶段任务

讨论区列表支持最新、热门和精华排序，并提供关键词搜索与新建帖子入口。帖子卡片同时展示作者、发布时间、课程或标签、浏览量、点赞数和评论数。课程问答或知识库内容经过整理后可以发布到讨论区，从个人学习记录转化为可交流、可检索的公共内容。

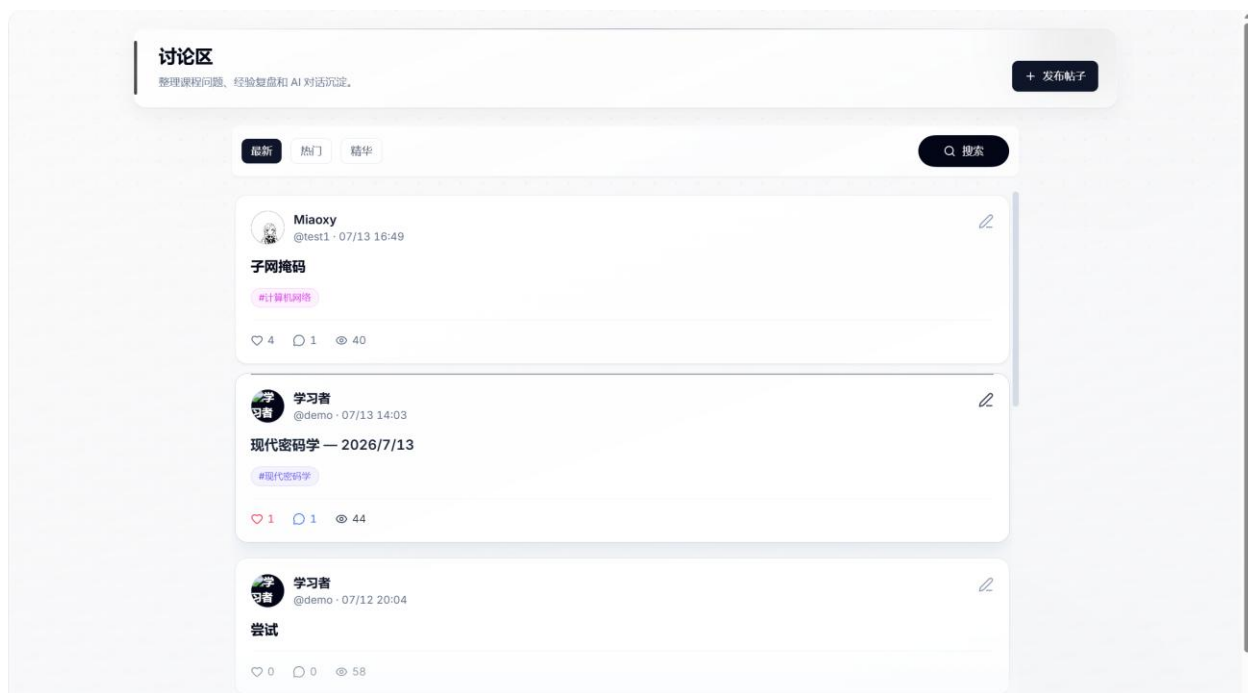


图 5-11 讨论区帖子列表

帖子详情页在主区域展示标题、作者、正文和评论，在右侧提供点赞状态与作者信息。评论输入、评论列表、编辑入口和返回讨论区操作保持在稳定位置。后端对帖子、评论和投票执行身份与所有权校验，前端在操作成功后精确刷新帖子和评论缓存。

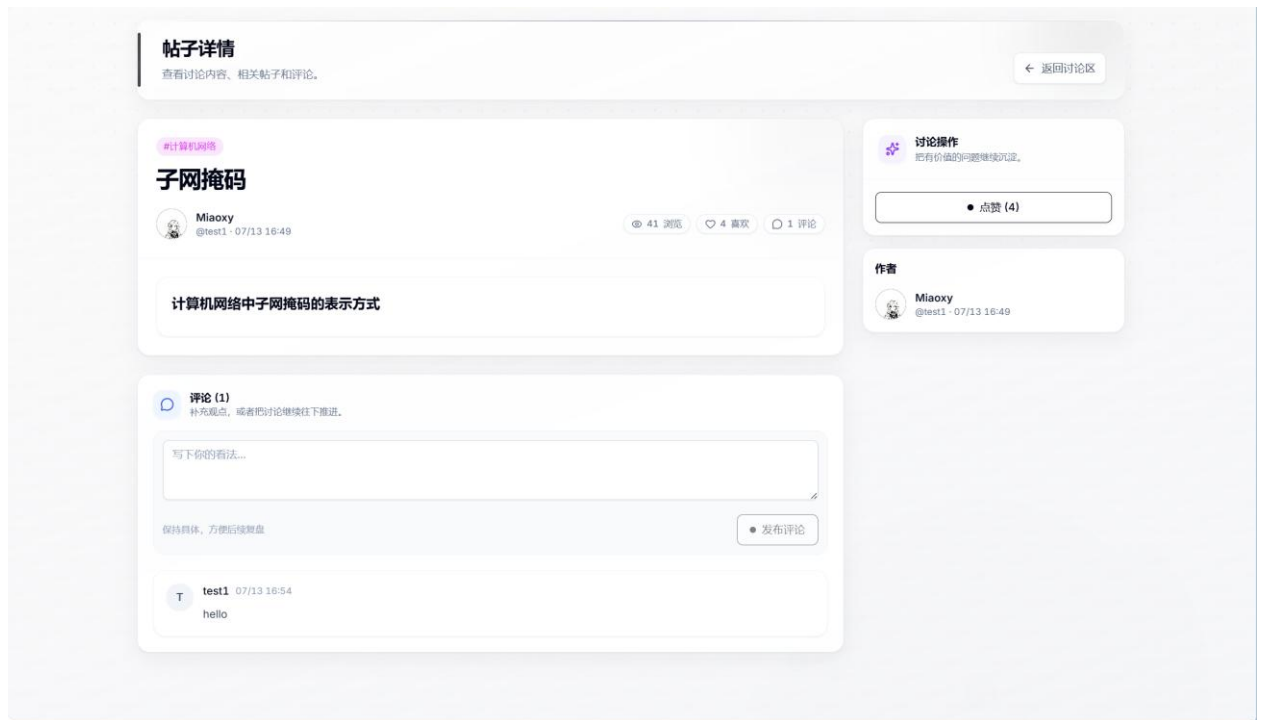


图 5-12 帖子详情、评论与作者信息

5.1.5 标签与个人学习空间

标签管理页汇总标签数量、已使用标签和关联内容，并提供搜索、创建、修改和删除入口。标签作为知识库与讨论区之间共享的分类维度，能够减少同一主题在不同模块中重复建立分类的问题。

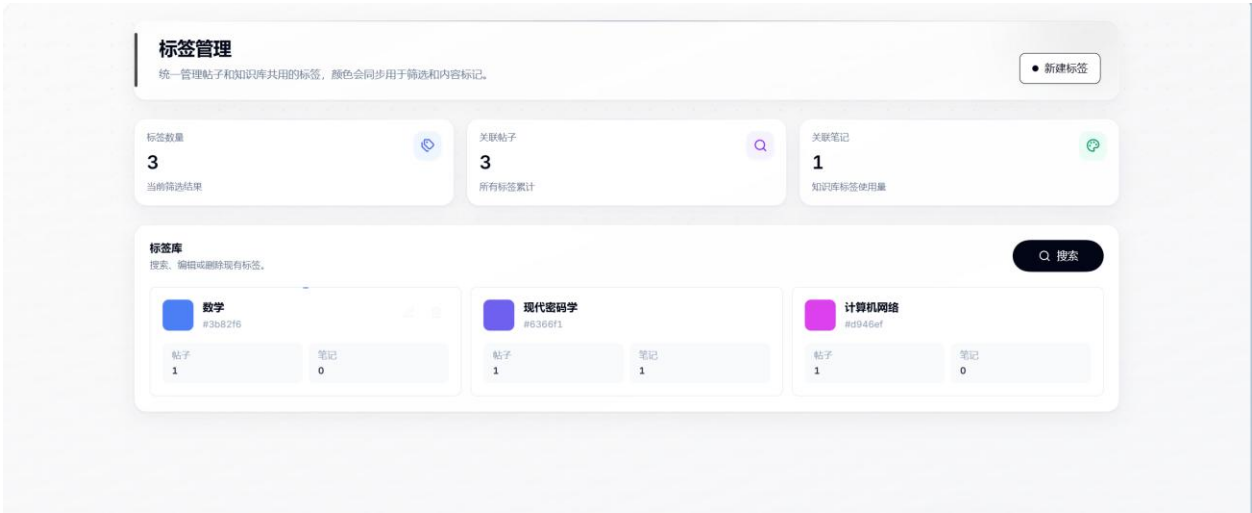


图 5-13 标签统计与标签管理

个人中心汇总用户头像、昵称、课程、任务、计划和学习活动。用户可以修改个人资料，并从课程、计划、会话等分区快速回到最近内容。该页面既承担账户资料维护，也承担个人学习档案入口，使分散在各业务模块中的数据能够按用户重新聚合。

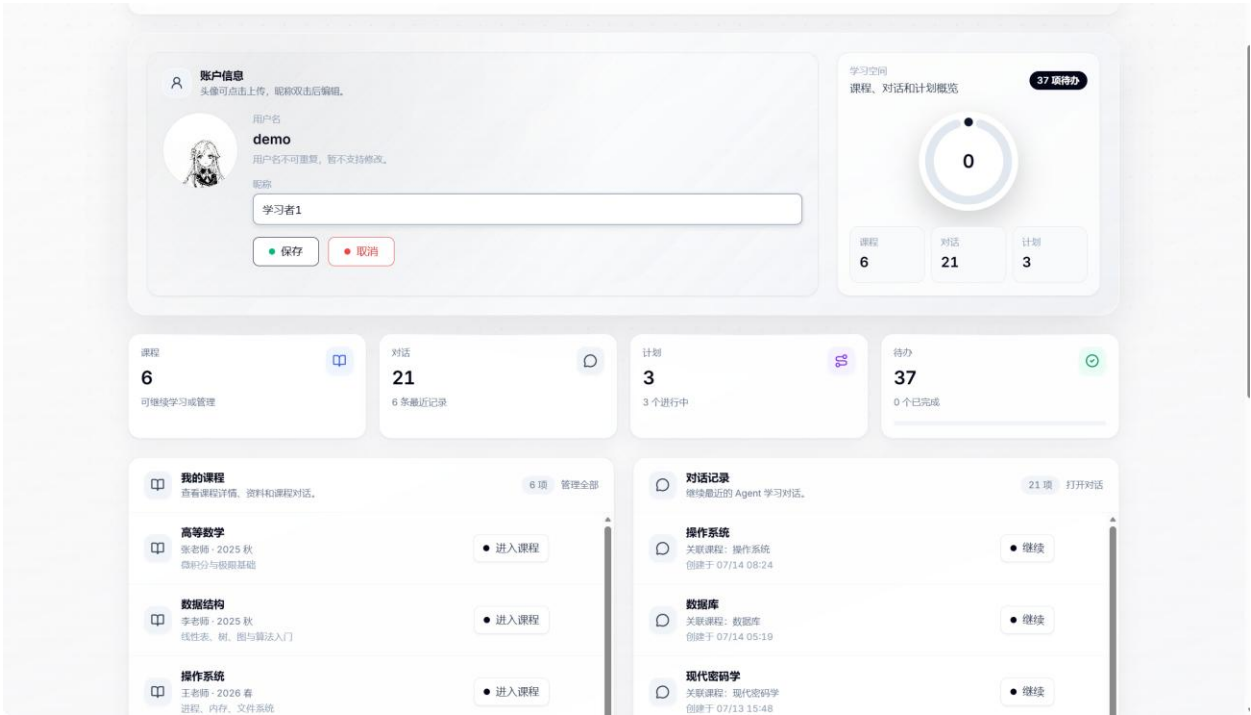


图 5-14 个人中心与学习数据汇总

以上运行结果表明，系统已经把课程资料、智能问答、知识沉淀、任务执行和讨论交流组织为连续流程。页面设计没有依赖单纯装饰，而是通过统一导航、状态反馈、搜索筛选、弹窗表单、骨架屏和响应式网格支撑真实操作。所有截图均来自当前运行版本，可与后续构建结果和功能测试记录相互对应。

5.2 测试环境与方法

项目	环境或命令	结果
前端类型检查	TypeScript 5.5, tsc -b	通过
前端生产构建	Vite 5.4, npm run build	通过，9116 个模块完成转换（本次构建 15.16 s）
前端分包	Vite manualChunks	页面与 vendor chunk 分离，无大 chunk 警告
后端测试	pytest -q	4 passed, 1 warning 耗时 19.06s

项目	环境或命令	结果
接口文档	FastAPI /docs	路由和 Schema 可自动展示
数据库	SQLite 本地 / PostgreSQL 配置	引擎按 URL 切换

2026 年 7 月 14 日重新执行了 `npm run build` 与 `python -m pytest -q`。前端构建成功，共转换 9116 个模块，最大的 Markdown vendor chunk 为 432.12 kB，其余核心框架、动画、路由、Query、图标和页面均已拆分；后端 4 项测试全部通过，耗时 19.06 s，覆盖服务根接口、注册登录与课程、自然语言计划与七日综合日程、资料引用片段结构。

5.3 功能测试用例

编号	测试内容	预期结果	状态
01	使用正确账号登录	返回 JWT 并进入受保护页面	自动测试
02	访问当前用户信息	返回用户名、昵称和头像	通过
03	创建并读取课程	课程归属当前用户且列表可见	自动测试
04	上传课程资料	文件落盘并生成 Material 记录	人工验证
05	下载他人资料	无法通过对象编号越权访问	代码审查
06	课程会话发送问题	保存消息并返回模型回答或明确错误	人工验证
07	创建计划并生成任务	任务按阶段生成并关联计划	人工验证
08	勾选任务完成	状态更新，仪表盘缓存失效	人工验证
09	知识节点删除与恢复	先进入回收站，再恢复到可见树	人工验证
10	发布帖子、评论与点赞	计数和作者信息正确更新	人工验证
11	侧栏切换懒加载页面	显示骨架后正常渲染，无空白页	人工验证
12	断开 API 或令牌过期	显示统一错误并跳转登录	人工验证
13	服务根接口健康检查	返回平台名称和版本信息	自动测试
14	自然语言计划与综合日程	自动推导截止日期，并返回 7 天日程	自动测试
15	资料引用片段数据结构	课程、文件名和块编号字段正确	自动测试

5.4 构建结果分析

构建日志显示页面已按路由独立输出，例如 Dashboard、Profile、Courses、ForumList、KBList、Chat、Plan、Login、CourseDetail 和 KBDetail 均为单独 chunk。

React、Router、Query、动画、Markdown、图标和上传依赖被拆入 vendor 文件，浏览器在多页面切换时可以复用缓存。与把所有依赖打入单文件相比，该方案提高首次进入轻量页面的效率，也使后续版本更新只需重新下载发生变化的 chunk。

5.5 当前限制与后续计划

限制	影响	改进方向
LLM 调用为同步请求	长回答可能占用请求线程	异步 httpx、流式响应、超时与重试策略
轻量运行时 schema 兼容	复杂迁移不可回滚	引入 Alembic 版本迁移
资料上传限制不完整	超大或异常文件存在风险	大小限制、文件名清洗、MIME 二次验证与病毒扫描
自动测试覆盖较少	知识库和论坛回归依赖人工	补充 Router、Service、权限和前端端到端测试
外部知识工具仅有验证入口	同步能力尚不完整	实现增量同步、冲突处理和凭据加密

后续迭代应优先补强安全和测试，而不是继续增加视觉组件。建议先完成上传边界、数据库正式迁移、知识库与论坛的权限测试，再引入流式回答和向量检索。前端继续遵循“交互必须能完成任务”的原则，新增动画前明确它表达的状态、空间关系或操作结果。

六、设计心得

6.1 从页面实现到系统设计

本次实践最直接的体会是，能够显示页面不等于完成系统。课程卡片、计划卡片和知识节点只有与稳定的数据模型、明确的接口状态码、权限校验和错误反馈连接后，才构成可用功能。早期如果只在前端模拟数据，后续接入数据库时会暴露字段命名、可空值、删除语义和缓存同步等大量问题。因此应在设计阶段就把用户操作写成端到端数据流。

REST API 的价值也不只是 URL 规范，而是建立团队共同理解。GET、POST、PUT、PATCH、DELETE 的语义、201/204/401/404 的状态、Pydantic Schema 和前端 `ApiError` 共同组成契约。只要契约稳定，前端和后端可以并行开发；契约模糊时，即使各自代码都能运行，集成仍会不断返工。

6.2 对 AI 功能的认识

把大模型接入应用并不困难，困难的是让它在正确的上下文、权限和失败边界内工作。课程资料增强问答需要同时考虑文件解析、内容长度、相关性、历史消息、来源标注、网络异常和成本。通过实现关键词检索版本，可以清楚观察每个步骤的输入输出，为后续替换向量检索留下基线，而不是一开始就引入不可解释的复杂组件。

模型生成的计划任务和摘要不能被视为事实。系统应把它们定位为建议，保留用户修改与删除权；模型无资料时应明确说明，不能伪造引用。实践使我们认识到，AI 功能的质量不仅取决于模型能力，更取决于数据准备、提示约束、界面反馈和人工确认机制。

6.3 对工程协作的认识

多人和多个编码 Agent 同时参与时，最重要的是让上下文进入仓库。README 负责运行和架构，AGENTS.md 与 CLAUDE.md 负责修改边界和验证标准，.gitignore 负责排除密钥、数据库、上传目录和构建产物。提交应按功能拆分并说明影响，合并前执行构

建和测试。规范不能代替沟通，但能把重复提醒转化为可执行检查。

最后，界面优化需要持续回到用户任务。强悬停、粒子、堆叠卡片和动态文件夹都可能吸引注意，但只有在它们帮助定位课程、表达选择状态或进入详情时才值得保留。克制的层级、稳定的间距和明确的反馈比装饰数量更能体现专业性。

七、团队分工说明

本项目由三位成员通过 Git 共同完成。以下分工根据仓库提交记录归纳，实际开发中存在交叉修改、代码评审和冲突解决，不以单一文件作为个人成果边界。提交前应由团队再次核对成员真实姓名、学号及工作比例，并补填封面信息。

成员标识	主要工作	协作与交付
苗新运	前端整体视觉与交互、登录页、侧栏、课程/仪表盘/计划/个人中心页面优化、性能与错误处理、开发文档	维护界面规范，清理冗余交互与用户体验优化，组织 README 与 Agent 协作规则
赵 亮	知识库与讨论区、数据库模型与兼容、课程对话、资料读取与 AI 引用、前后端接口集成	处理知识库结构和前后端契约，完善检索与模型服务
孙创辉	仪表盘、学习计划、个人学习空间、认证安全与测试增强、资料检索功能	管理流程、计划生成、身份验证和冒烟测试，删除纠正，向量检索功能的升级

7.1 协作流程

1. 开始任务前拉取最新分支，检查工作区和目标文件，避免覆盖他人未提交修改。
2. 前端、后端或全栈任务先确定边界；字段与接口变化必须同时更新 client、server 和文档。
3. 实现过程中保持补丁小而可审查，优先复用现有组件、Hook、Router 和 Schema。
4. 提交前执行与改动相关的最小验证：前端运行 npm run build，后端运行 pytest。
5. 通过 Git 提交记录、README、AGENTS.md 和 CLAUDE.md 传递设计决策，冲突由成员共同确认。

7.2 分工评价

团队工作覆盖了界面、业务、数据、智能服务和质量保障。项目后期多次围绕冗余信息、无意义动效、路由空白、数据库连接和 AI 引用进行联合修正，说明成果来自持续集成而非简单模块拼接。当前仍需通过更多自动测试和正式迁移工具降低合并后的回

归风险。

八、AI 伦理声明

8.1 AI 使用范围

本项目在需求梳理、代码生成建议、界面方案比较、故障排查、文档整理和报告草拟过程中使用了 Codex、Claude Code 等生成式 AI 工具；运行时智能问答使用 DeepSeek 的 OpenAI 兼容接口。AI 工具作为辅助，不替代团队成员对需求、代码、测试结果和最终提交内容的判断。所有进入仓库的代码应由成员阅读、构建或测试后确认。

8.2 数据与隐私原则

- 不把 .env、数据库密码、API Key、用户令牌和未公开个人资料提交到代码仓库或写入报告。
- 模型密钥仅由后端环境变量读取，浏览器端不直接接触 DeepSeek API Key。
- 课程问答只检索当前会话关联且当前用户有权访问的课程资料。
- 发送给模型的上下文应限制数量与长度，避免无关资料进入外部服务。
- 用户上传资料可能涉及版权和隐私，部署时应提供用途说明、删除机制和访问控制。

8.3 可靠性与学术诚信

模型回答可能存在事实错误、遗漏和虚假推断。系统通过来源提示、无资料说明和错误降级降低误导，但这些措施不能保证答案绝对正确。用户应把回答视为学习辅助，并回到课程材料和权威资料核验。自动生成的复习提纲、任务和报告文本必须经过人工校对，不得冒充未经验证的实验数据。

本报告内容基于当前仓库源代码、README、Git 提交记录以及实际执行的构建和测试结果整理。团队成员对报告最终准确性、引用规范和提交责任负责；AI 生成内容若与源代码不一致，应以可运行实现和人工核查结果为准。

8.4 公平、透明与可控

原则	项目落实方式
透明	说明模型提供者、资料来源和 AI 辅助范围
可控	用户可修改或删除计划、任务、帖子和知识内容
最小化	仅发送回答所需的少量相关片段，不默认上传全部资料
可追溯	保存会话历史、知识修订和引用文件名
责任	构建、测试与最终提交由团队成员确认

参考资料

- [1] React Documentation. <https://react.dev/>
- [2] React Router Documentation. <https://reactrouter.com/>
- [3] TanStack Query Documentation. <https://tanstack.com/query/>
- [4] Vite Documentation. <https://vite.dev/>
- [5] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [6] SQLAlchemy Documentation. <https://docs.sqlalchemy.org/>
- [7] Pydantic Documentation. <https://docs.pydantic.dev/>
- [8] DeepSeek API Documentation. <https://api-docs.deepseek.com/>
- [9] 项目仓库 README.md、AGENTS.md、CLAUDE.md 及当前源代码。

附录 A：主要目录结构

```
client/

src/api/          # 统一 API 客户端

src/components/   # 布局、反馈、骨架屏和通用组件

src/hooks/        # TanStack Query 与草稿 Hook

src/pages/        # 课程、对话、计划、知识库、论坛等页面

src/router.tsx    # 路由、懒加载与错误边界

server/

app/routers/      # auth、course、chat、forum、kb 等 REST Router

app/services/     # LLM 与资料检索

app/models.py     # SQLAlchemy 数据模型

app/schemas.py   # Pydantic 请求/响应模型

app/database.py   # 数据库引擎与兼容逻辑

tests/           # 后端冒烟测试
```

以上目录把页面展示、网络请求、服务逻辑、接口路由和持久化模型分开，便于团队按领域定位修改。新增功能时应先确定所属模块，再同步调整 Schema、API 客户端、Query Hook 和页面，避免业务逻辑散落。